

Programação 1

Relatório do trabalho

2023/2024

Ouri



Trabalho realizado por:

- Diogo Tavares nº 58049
- Rodrigo Barreto nº 58023
- Cristiano Cascarrinho nº 58191

Índice

Introdução	3
ouri.h	4
Funções	4/5/6
Dificuldades Encontradas	7
Conclusão	9

Introdução:

Este trabalho prático tem como objetivo demonstrar os conhecimentos lecionados no âmbito da disciplina de Programação 1 por meio da realização do jogo de tabuleiro ouri através da linguagem C. Este jogo de tabuleiro de 2 jogadores consiste em tentar capturar as pedras do seu adversário e não deixar que este capture as suas. O Ouri é jogado num tabuleiro de quatorze casas , seis para cada jogador e duas para os depósitos de cada jogador, que servem para armazenar as pedras capturadas. O jogo acaba, não só mas maioritariamente, quando algum jogador alcança 25 pedras no seu depósito.

ouri.h:

O header ouri.h contém as livrarias e informações necessárias para correr o código e variáveis que são usadas em várias funções do código.

Funções:

- void guardarTabuleiro(int tabuleiro[14], char *nomeArquivo);
- int carregarTabuleiro(int tabuleiro[14], char *nomeArquivo);
- void escolha(char *pvp)void escolha(char *pvp);
- int fim(int tabuleiro[14]);
- void imprimirTabuleiro(int tabuleiro[14])
- void jogada(int tabuleiro[14]), int jogadoratual, int pvp)
- int main(int argc, char *argv[])

- **void guardarTabuleiro(int tabuleiro[14], char *nomeArquivo):**

A função é utilizada para guardar um ficheiro com a informação sobre o tabuleiro no estado atual para que a função carregarTabuleiro a possa abrir futuramente, esta função tem que receber a informação do tabuleiro. Como abrimos o arquivo com “w” podemos escrever no arquivo e realizamos esta tarefa com a ajuda do fprintf, ajustando o formato do arquivo ao formato pedido.

Os argumentos presentes nesta função são o array tabuleiro e o nomeArquivo.

O argumento tabuleiro[14] é um array em que as suas posições são casas do tabuleiro, sendo as casas de 0 a 5 as casas do jogador 1, as casas de 6 a 11 as casas do jogador 2, e as posições 12 e 13 os depósitos dos jogadores 2 e 1, respetivamente.

O argumento nomearquivo é utilizado para nomear o arquivo guardado.

Também sabemos que como a função serve para guardar informações esta não precisa de ter nenhum return logo é uma função void.

- **carregarTabuleiro(int tabuleiro[14], char *nomeArquivo):**

Esta função serve para carregar o ficheiro guardado pela função guardarTabuleiro. Assim, esta função usa o fscanf para ler os valores do ficheiro com as informações do tabuleiro e afetar os valores correspondentes no tabuleiro.

Os argumentos da função são o nomeArquivo guardado e os valores do tabuleiro.

A função retorna o valor 1 se for possível abrir o arquivo, caso contrário retorna 0.

- **void escolha(char *pvp)**

A função void escolha serve apenas para afetar a variável pvp, que será utilizada para saber se o jogador quer jogar contra outro jogador ou contra um computador, logo tem apenas como entrada a variável pvp

O argumento da função é o pvp, que serve para para a escolha do adversário, que será outro jogador ou o computador.

Como a função é void, não retorna nenhum valor.

- **int fim(int tabuleiro[14]):**

A função int fim é utilizada para ver se o jogo acabou ou não, na função há uma verificação dos valores dos depósitos, em que o jogo acaba quando um dos depósitos chega a 25 ou os dois ficam com 24 pontos, acabando empatados.

O argumento da função é o tabuleiro[14].

A função retorna 1 se o jogo acabar e 0 se o jogo ainda não tiver acabado.

- **void imprimirTabuleiro(int tabuleiro[14]):**

A função imprimir Tabuleiro serve só para imprimir o tabuleiro no formato correto no fim de cada jogada, precisa apenas dos valores das pedras de cada casa.

O argumento da função é o tabuleiro[14].

Como a função é void, não retorna nenhum valor.

- **void jogada(int tabuleiro[14]), int jogadoratual, int pvp):**

A função void jogada é onde se passa a maior parte do jogo, já que esta função é responsável pelas jogadas, capturas e algumas regras. As regras implementadas nesta função são as regras dos movimentos, de capturas e as regras complementares.

Os argumentos presentes nesta função são: um array que representa o tabuleiro, o jogador atual e o PVP.

O argumento tabuleiro[14] é um array em que as suas posições são casas do tabuleiro, sendo as casas de 0 a 5 as casas do jogador 1, as casas de 6 a 11 as casas do jogador 2, e as posições 12 e 13 os depósitos dos jogadores 2 e 1, respetivamente.

O argumento jogador atual devolve 0 ou 1. Mas nós adicionamos 1 ao valor para se tornar jogador 1 ou jogador 2.

O argumento PVP é utilizado para distinguir se o jogador vai jogar contra outro jogador ou contra o computador.

Como a função é void não retorna nenhum valor.

- **int main(int argc, char *argv[]):**

A função main é a parte inicial do programa, é responsável por inicializar o jogo, permitindo ao jogador escolher o modo de jogo (PvP ou PvE). Exibe o estado atual do tabuleiro. Ao final do jogo, imprime o estado final do tabuleiro e retorna 0, indicando uma conclusão bem-sucedida do programa. Também é responsável pela função de carregar posições do tabuleiro a partir de ficheiros em formato de texto.

O argumento argc indica o número total de argumentos passados para o programa, incluindo o próprio nome do programa.

O argumento argv é um array que recebe os argumentos passados para o programa.

A função retorna 0 quando o jogo for encerrado.

Dificuldades encontradas:

- Após implementar a lógica de captura na função “void jogada” encontramos um erro que fazia com que o jogador pudesse capturar as próprias pedras se a sua jogada fizesse com que caíssem pedras em casas suas formando assim casas com 2 ou 3 pedras. Após tentarmos arranjar este erro deparamo-nos com o facto de que se, por exemplo, se o jogador 1 iniciar uma captura no lado do tabuleiro do jogador 2 e essa captura fizesse uma reação em cadeia de mais capturas (pois as casas seguintes do jogador 2 atendiam às condições para continuar a capturar) e essa cadeia regressar até ao tabuleiro do jogador 1 (o jogador que iniciou a reação de capturas) a cadeia de capturas iria também capturar as peças do jogador 1 caso a casa que elas estivessem atendessem às condições para tal (terem 2 ou 3 pedras dentro das casas). Para resolver esse problema implementamos restrições dentro da lógica de captura que se encontra dentro da função jogada, essas restrições foram feitas dentro do while para que ele parasse a cadeia assim que entrasse no lado do tabuleiro do jogador que iniciou a captura.
- Tivemos dificuldade em implementar a regra que fazia com que o jogador não pudesse jogar casas com 1 pedra se tivesse casas com mais de 1 pedra no seu lado do campo. Nas primeiras tentativas não conseguimos implementar a regra de todo e o jogador podia livremente jogar casas com 1 pedra, nas próximas tentativas arranjamos este problema mas encontramos outro pois se 1 jogador tivesse casas com mais de 1 pedra o outro não conseguia jogar as suas casas com 1 pedra mesmo se essas fossem as únicas que ele tinha. Resolvemos o problema substituindo as variáveis que verificavam estas regras pois as antigas variáveis apenas verificavam a casa anterior e posterior ou então o tabuleiro todo,

mas removemos estas variáveis e colocamos manualmente cada casa do tabuleiro e uma condicional para separar os jogadores e os seus respectivos lados desta regra, de maneira a que se um jogador tivesse casas com mais de 1 pedra o outro não fosse afetado e assim pudesse jogar o jogo como é devido, cumprindo a regra que diz que se o jogador tiver casas com mais de 1 pedra no seu lado do tabuleiro este não pode jogar essa casa.

- Ao implementar a regra que diz que se o jogador ao jogar uma casa com 12 ou mais pedras a distribuição das pedras deve pular a casa que foi jogada, tivemos o problema de se o jogador jogar uma casa que tivesse pedras para dar múltiplas voltas ao tabuleiro esta regra não funcionaria na 2ª volta. Também tivemos o problema de que se o jogador devido a esta regra jogasse uma casa de 12 ou mais pedras esta iria pular a casa jogada mas as jogadas consequentes também iriam pular esta casa até ao fim do jogo.
- Não achamos nenhuma maneira que não compromete-se o código de maneira a que pudéssemos forçar o jogador a jogar uma casa que colocasse pedras no campo adversário caso o adversário não tivesse casas no seu campo, por isso usámos um printf para avisar o jogador que tem casas no seu campo que teria de fazer tal jogada por esta mesma razão. Mas reconhecemos que fazendo isto, quando estamos a jogar contra o computador este não consegue ler a mensagem logo poderá fazer jogadas que não cumpram esta regra, o que poderá comprometer o jogo.

Conclusão:

Durante a realização do trabalho sentimos que ao juntarmos vários conhecimentos previamente lecionados para situações específicas num só objetivo descobrimos muitas mais maneiras de utilizar e simplificar o nosso código. Concordamos mutuamente também que aprendemos bastante sobre a linguagem C, as suas livrarias e as suas limitações, para além de esclarecermos e pôrmos em prática muitas dúvidas que tínhamos sobre a utilização da matéria lecionada no âmbito da disciplina de Programação 1.